
**KULLIYYAH OF ENGINEERING**

**MCTA 4363 DEEP LEARNING**

**SECTION 1**

**SEMESTER 2, 2025/2026**

**GROUP NAME: FIGHTER 3.6**

**PROJECT TITLE: FlagOS:High Performance Triton Operator Development**

**TEAM MEMBER**

| NAME | MARIC NO. |
|---|---|
| MUHAMMAD IKHWAN BIN MUHAMAD FAUZI | 2218845 |
| SYABAB ALHAQQI BIN JAAFAR | 2211117 |
| ZAMIR MUTTAQIN BIN MUHAMMAD BALYAN | 2212985 |

**LECTURER: ASSOC. PROF. TS. DR. MOHD HASAN FIRDAUS BIN MOHD ZAKI**

Date of Submission: Thursday, 7 May 2026

## ASSESSMENT 2: OPERATOR OPTIMIZATION AND METHODS

FlagGems Operator Development Competition - Track 1

### 1. Introduction and Objectives

Our primary goals were to address performance bottlenecks identified in Assessment 1 and expand our operator coverage. Specifically, we focused on:

Optimizing the median operator: The baseline implementation achieved only a ~0.3x speedup compared to PyTorch because it relied on full sorting (Bitonic and Radix Sort). We transitioned to a highly optimized selection algorithm to surpass the required $\geq$ 0.9x speedup threshold.

### 2. Techniques Tried

Median Optimization: From Sorting to Selection

1. Full Sorting (Baseline): Initially used Bitonic and Radix Sort O(N log N). Failed to achieve speedup due to the overhead of sorting every element when only rank K = (N-1)/2 is needed.
2. QuickSelect (Prototype): Attempted an O(N) QuickSelect. Abandoned due to high thread divergence and uncoalesced memory accesses on the GPU SIMT architecture.
3. Radix Select in Registers (Final medianV2): Developed a strictly register-bound O(N) Radix Select. It loops over the 32 bits of a monotonic integer representation of the floats, pruning active lanes based on pop-counts. This completely eliminated HBM round-trips for intermediate bins and achieved speedups of ~1.1x - 1.5x over PyTorch.
4. Edge Case Canonicalization (V3): During extensive unit testing with duplicates, we discovered discrepancies between PyTorch's native high-level float logic and our bit-level sorting. Specifically, floats like -0.0 and 0.0 evaluate equally in Python, but their integer bit representations differ. Additionally, NaN values carry varying bit payloads. We patched the kernel by standardizing -0.0 to 0.0 and mapping all NaNs to a canonical positive NaN pattern before bitcasting to uint32. This ensured 100% stable matching with PyTorch's native subset evaluation.

### 3. Insights Gained

Pass Efficiency over Algorithmic Asymptotics: For GPU kernels (like our medianV2), minimizing the number of passes over data in registers drastically reduces instruction counts and latency, even if the theoretical complexity involves a higher constant factor (32 bit passes).

### 4. Performance Results

The following benchmarks demonstrate the speedup of our optimized median.dim (Radix Select) implementation compared to the native PyTorch CUDA baseline:

| Shape | Dimension | Triton (μs) | PyTorch (μs) | Speedup |
|---|---|---|---|---|
| (1024, 64) | 1 | 46.2 | 61.5 | 1.33x |
| (1024, 512) | 1 | 112.4 | 168.1 | 1.49x |
| (4096, 64) | 1 | 158.9 | 192.3 | 1.21x |

## 5. Implementation Status

The optimized operators have been submitted to the official FlagGems repository:

Median.dim (Radix Select): Submitted via Pull Request [FlagGems Operator Development Competition] Add median.dim (Radix Select) operator.

The complete code, iterative development log, and test notebooks for detailed execution logs and accuracy are available in this repository:

- Core Implementation: src/flag_gems/ops/medianV2.py
- Development Log: assessment2_dev_log.md
- Verification Notebook: flagos_median3.ipynb

## 6. Future Work

For assessment 3, our future plans consist of:

1. Improving our median operator
2. Try difficult level operator such as ctc_loss